



GreenPulse – Smart Agentic IoT Plant-Care Device with LLM Backend

Assignment released on	25th August 24, 2026
Due date	20th September 2026
Total	100

- This is a group assignment with a minimum of two and a maximum of four members in each group. However, your performance will be assessed individually.

Scenario

You have been recruited to design and develop a smart IoT system named GreenPulse. This system combines environmental sensor data, intelligent summarization, and AI-generated care messages to help users keep their indoor plants healthy. GreenPulse integrates IoT hardware, secure cloud-based MQTT communication via AWS IoT Core (or an equivalent managed MQTT broker such as Azure IoT Hub or HiveMQ Cloud), a cloud-deployed server that runs the LLM backend, and a Node-RED dashboard for monitoring sensor data and MQTT message payloads.

Objectives

- Build a smart IoT device with ESP32 that senses plant and indoor growing conditions.
- Use a cloud MQTT broker (AWS IoT Core, Azure IoT Hub, or HiveMQ Cloud) for secure, TLS-based communication between device and backend.
- Develop a backend using an LLM framework (e.g., LangChain with OpenAI API).
- Deploy the LLM backend on a cloud-based server and create a Node-RED dashboard that displays live sensor readings together with the LLM-related MQTT publish and subscribe payloads.
- Display only live sensor readings and MQTT connection status on the IoT device's LCD/OLED; present LLM-generated care messages, alerts, and MQTT payloads on the Node-RED dashboard.
- Indicate watering/care urgency using LEDs or display cues.
- Encourage creativity through extensions.

Tasks

Part 1: IoT Device (ESP32)

1. Attach at least a soil-moisture sensor and a temperature/humidity sensor (e.g., capacitive soil moisture sensor + DHT22 or BME280).
2. Program the ESP32 to read sensor values and publish them securely via MQTT over TLS to a cloud MQTT broker (AWS IoT Core is recommended; Azure IoT Hub or HiveMQ Cloud are acceptable alternatives).
3. Subscribe to relevant backend MQTT messages, but display only real-time sensor readings and MQTT connection status on the device's LCD/OLED/TFT. Do not display LLM-generated messages or MQTT payloads on the device screen.
4. Implement a priority indicator (e.g., RGB LED or backlight) showing watering or care urgency.

Part 2: Backend with Agentic AI (LLM)

1. Deploy the backend on a cloud-based server capable of running the LLM application. Connect it to the same cloud MQTT broker used by the device (AWS IoT Core / Azure IoT Hub / HiveMQ Cloud), subscribe to sensor data, and log readings (e.g., to DynamoDB, S3, or a database).
2. Use an LLM framework to: generate a literature-style quote or care message inspired by the plant's current soil and indoor conditions; and connect to a real weather API and the student's real notification/email account (via Gmail API, Outlook API, or IMAP) to summarize watering reminders, weather alerts, or gardening-related messages.
3. Publish the generated care quote and summarized alerts back to the IoT device through the cloud MQTT broker, using clearly defined MQTT topics and JSON payloads.
4. Create a Node-RED dashboard that displays live sensor data, the LLM input/subscribed payload, the LLM output/published payload, relevant MQTT topic names, timestamps, and connection status.

Part 3: Optional Extensions (Extra Marks)

- Add predictions (e.g., forecast when the plant will next need watering).
- Integrate extra sensors (light/lux, CO₂, air quality, motion, etc.).
- Use a multi-agent design (separate agents for environment, weather, and care-priority).
- Add creative outputs (e-paper display, voice alerts, smart-lamp integration).
- Enable user interaction (e.g., "GreenPulse, does my plant need watering today?").
- Use AWS IoT Core Rules Engine (or equivalent) to trigger Lambda functions/automations directly from incoming sensor data.

Deliverables

1. Source Code and Deployment: ESP32 firmware, cloud-hosted LLM backend code, Node-RED flow/export, and MQTT broker/cloud-server configuration with deployment instructions.
2. Demonstration: Live end-to-end demo showing the ESP32, cloud MQTT communication, cloud-deployed LLM backend, and Node-RED dashboard displaying sensor data and the LLM subscribed and published payloads. The device LCD/OLED must show only sensor readings and MQTT connection status.
3. Commercializing product video.
4. Report (4–6 pages): architecture diagram including the device, cloud MQTT broker, cloud-based LLM server, and Node-RED dashboard; hardware/software and deployment details; MQTT topics and payload formats; explanation of LLM integration for care-tip generation and weather/notification summarization; reflection on challenges and creative features.

GreenPulse Project – Rubric-Style Marking Guide

Criteria	Poor	Fair	Good	Excellent	Marks
Part 1: IoT Device (30 marks)					
Sensor Integration (10)	No/incorrect data; unstable	Reads partially but unreliable	Stable readings from one sensor	Accurate, reliable readings from multiple sensors (soil moisture + DHT22/BME280)	/10
MQTT Communication via AWS IoT Core (5)	No MQTT/TLS connection or fails	Connects intermittently, insecure	Stable one-way communication with certificate-based auth	Reliable bidirectional publish/subscribe over TLS with AWS IoT Core policies	/5
LCD Display Functionality (10)	No display or incorrect information	Displays limited sensor readings or connection status	Displays sensor readings and MQTT status with minor issues	Clearly displays only live sensor readings and MQTT connection status	/10
Priority Indicator (5)	No indicator	Limited LED feedback	Functional LED with basic urgency	Clear urgency mapping with RGB/backlight cues	/5
Part 2: Backend with Agentic AI (40 marks)					
Cloud LLM Server & Data Handling (5)	No cloud deployment or backend logging	Limited or unreliable cloud deployment and logging	Cloud-deployed LLM backend with consistent sensor-data logging	Reliable cloud deployment with secure configuration and robust timestamped logging	/5
LLM Care-Tip Generation (10)	No AI use	Basic text, low creativity	Context-aware tips, somewhat botanical/literary	High-quality, creative, plant-specific care advice and quotes	/10
Weather & Notification Summarization (15)	No/poor summarization	Limited clarity, inaccurate	Clear summaries, moderate accuracy	Accurate, concise, context-rich summaries of weather + gardening alerts	/15
MQTT Response & Node-RED Dashboard (10)	No MQTT response or dashboard	Partial MQTT response; dashboard shows limited data	Reliable MQTT messaging; dashboard shows sensor data and some LLM payload details	Stable bidirectional MQTT; clear live sensor data, topics, and LLM input/output payloads	/10
Part 3: Optional Extensions (15 marks)					
Creative Extensions (predictions, extra sensors, agents, voice/e-paper, interaction)	None attempted	Attempted but minimal	Functional, adds value	Polished, highly creative, enhances system significantly	/15
Deliverables (15 marks)					
Source Code & Deployment (5)	Incomplete code; no cloud deployment or Node-RED flow	Partial code/deployment files	Functional code, cloud deployment, and Node-RED flow with minor issues	Clean, complete, well-documented code, deployment instructions, and Node-RED export	/5
Demonstration (5)	No/failed demo	Partial demo	Mostly functional end-to-end demo including the dashboard	Smooth end-to-end demo of device, cloud LLM server, MQTT, and Node-RED payload visualization	/5
Report (5)	Missing/unclear	Basic coverage	Clear architecture and deployment coverage with some analysis	Excellent architecture, MQTT topic/payload, cloud deployment, dashboard, and reflection details	/5